**System Design: Solar Panel and Battery System**

EECE 2140 Computing Fundamentals for Engineers, Iteration 2

Advaith Nair, Jailen Patel

October 23, 2026

**Team change:** Teammate 3 left the group during Iteration 1, and all of their tasks are now split evenly between us. Advaith took the LoadProfile class, the sample input files, and keeping GitHub Issues up to date, since these connect to the input reader and repository he already owns. Jailen took the Report class, the README and iteration documents, and the meeting notes, since these connect to testing and the project records.

This is the first group project for both of us, so a lot of this iteration went into learning how to work as a team: how to split the work, how to use Git branches without overwriting each other, and how detailed a task list needs to be. We are still figuring out many parts of this process, and we expect the plan to change as we learn.

In this iteration we wrote a first version of the system design, listed every task, and set up the program so it compiles and runs a menu with all ten tasks. Most tasks are still stubs that print "not implemented yet." We have started three of them (T1, T3, and T4), but none are finished.

# 1. Project Leader

Jailen Patel is the Project Leader for Iteration 2. Jailen has not led the project before. Advaith will take the role in a later iteration so that leadership rotates.

Neither of us has had a leader role on a team project before, so we are still working out what daily check-ins and status reports should look like. For now, Jailen will:

- Check in with Advaith each day after class or in the group chat on what was finished and what is blocked.
- Make sure tasks stay split fairly now that the team is two people.
- Keep the task table in the Excel project plan up to date when a task changes status.
- Present the status report at the check-in meeting.

# 2. Task List

Every feature of the program is a separate task with its own menu option. Each task has one owner, except the three tasks built on the EnergyManager, which we write together as planned in Iteration 1. Status values are not started, stub, in progress, and done. Right now the only finished item is the menu itself.

| ID | Task | Menu | Owner | Files and functions | Deliverable | Status |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| T1 | Load sunlight data | 1 | Advaith | input_reader.cpp: readHourlyCsv() tasks.cpp: taskLoadSunlight() solar_panel.cpp: setSunlight() | Reads and checks a sunlight CSV file | In progress |
| T2 | Load demand data | 2 | Advaith | input_reader.cpp: readHourlyCsv() tasks.cpp: taskLoadDemand() load_profile.cpp: setDemand() | Reads and checks a demand CSV file | Stub |
| T3 | Set up solar panel | 3 | Advaith | solar_panel.cpp: setSize(), energyAt(), totalEnergy() tasks.cpp: taskSetupPanel() | Sets panel size and efficiency, prints hourly output | In progress |
| T4 | Set up battery | 4 | Jailen | battery.cpp: setLimits(), charge(), discharge() tasks.cpp: taskSetupBattery() | Sets capacity, minimum charge, and rate limit | In progress |
| T5 | Run 24-hour simulation | 5 | Both | energy_manager.cpp: runSimulation(), checkBalance() tasks.cpp: taskRun24Hour() | Hour-by-hour routing for one day | Stub |
| T6 | Run 7-day simulation | 6 | Both | energy_manager.cpp: runSimulation() tasks.cpp: taskRun7Day() | Same routing over 168 hours | Stub |
| T7 | Print summary | 7 | Jailen | report.cpp: printSummary() tasks.cpp: taskPrintSummary() | Totals and share of demand met by solar, battery, and grid | Stub |
| T8 | Save hourly results | 8 | Jailen | report.cpp: writeHourlyCsv() tasks.cpp: taskSaveResults() | Hourly output CSV that opens in Excel | Stub |
| T9 | Compare two battery sizes | 9 | Both | energy_manager.cpp: runSimulation() report.cpp: printSummary() tasks.cpp: taskCompareBatteries() | Side-by-side summary for two capacities | Stub |

| | | | | | | |
|---|---|---|---|---|---|---|
| T10 | Run class tests | 10 | Jailen | tests.cpp: taskRunTests() | At least 3 passing tests per class | Stub |

These supporting tasks do not have a menu option but are needed to finish the project:

| ID | Task | Owner | Files | Status |
|---|---|---|---|---|
| S1 | Menu and switch | Advaith | main.cpp: main(), printMenu() | Done |
| S2 | Sample input files | Advaith | data/sunlight_sunny.csv, data/sunlight_cloudy.csv, data/demand_typical.csv, data/sunlight_week.csv | In progress (7-day file not started) |
| S3 | Task table and project plan | Jailen | docs/EECE2140_Project_Plan.xlsx | In progress (first version) |
| S4 | README and iteration report | Jailen | README.md, docs/Iteration2_Report.pdf | In progress |
| S5 | Integration and code review | Advaith | GitHub main branch | In progress |
| S6 | GitHub Issues upkeep | Advaith | GitHub Issues | In progress |
| S7 | Meeting notes | Jailen | docs/meeting_notes.md | In progress |

# 3. System Design

This is our first version of the design. We based it on the header files from Iteration 1, and we expect to add or change methods once we start writing the code inside them.

## 3.1 Classes

Each class represents one part of the system. Data members are private and are only changed through public methods, so for example the battery charge can only change through charge() and discharge(), which keep it between the minimum charge and the capacity.

| Class | Private data | Public methods | Owner |
|---|---|---|---|
| SolarPanel | areaM2, efficiency, sunlight (vector<double>, W/m^2 per hour) | setSize, setSunlight, getHours, energyAt(hour), totalEnergy, print | Advaith |
| Battery | capacityKWh, minChargeKWh, maxRateKW, chargeKWh | setLimits, charge(kWh), discharge(kWh), getCharge, getPercent, reset, print | Jailen |

| | | | |
|---|---|---|---|
| LoadProfile | demand (vector<double>, kWh per hour) | setDemand, getHours, demandAt(hour), totalDemand | Advaith |
| EnergyManager | results (vector<HourResult>) | runSimulation(panel, battery, load, hours), getResults, checkBalance(tolerance) | Both |
| Report | none | printSummary(results), writeHourlyCsv(results, filename) | Jailen |
| HourResult (struct) | hour, solarKWh, demandKWh, solarToHome, solarToBattery, solarToGrid, batteryToHome, gridToHome, batteryChargeKWh | none (plain data record for one hour) | Both |
| SimulationState (struct) | panel, battery, load, manager, report | none (passed by reference to every task function) | Advaith |

**Units:** sunlight is in W/m^2 averaged over each hour, demand and energy are in kWh per hour, and the battery rate is in kW. Panel output for one hour is sunlight / 1000 x area x efficiency.

## 3.2 Energy Routing (planned for EnergyManager)

None of this logic is written yet. Our current plan is that runSimulation() will follow the routing order from the project plan for each hour:

1. Use solar energy to cover the home demand first.
2. If solar is more than demand, charge the battery with the extra, up to its rate limit and capacity.
3. Send any solar that is still left to the grid.
4. If solar is less than demand, discharge the battery to cover the gap, down to its minimum charge.
5. Take whatever demand is still not covered from the grid.
6. Save the hour as one HourResult, including the battery charge at the end of the hour.

checkBalance() will then confirm that, for every hour, solar plus battery output plus grid input equals demand plus battery input plus grid output within 0.001 kWh.

## 3.3 Menu

The menu runs in a loop until the user picks 0. Each option calls exactly one task function through a switch statement. Invalid input, such as letters or a number outside 0 to 10, prints a message and shows the menu again.

```
===== Solar Panel and Battery Simulator =====
 1. Load sunlight data from CSV
```

2. Load household demand from CSV
3. Set up solar panel and show hourly output
4. Set up battery
5. Run 24-hour simulation
6. Run 7-day simulation
7. Print summary of last run
8. Save hourly results to CSV
9. Compare two battery sizes
10. Run class tests
0. Exit

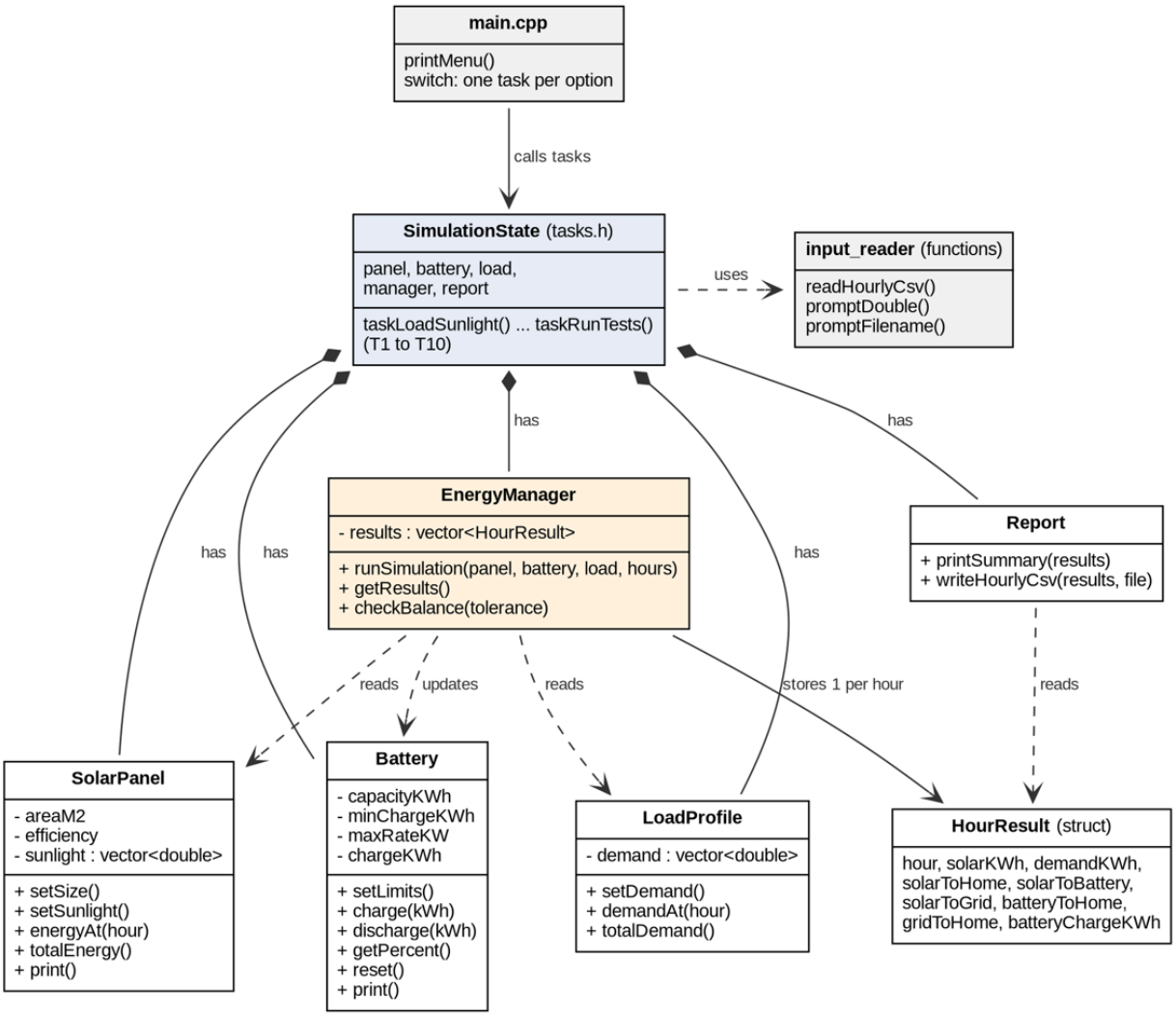
## 3.4 File Structure

| File | Contents | Owner |
|---|---|---|
| main.cpp | main(), printMenu(), the switch statement | Advaith |
| solar_panel.h / .cpp | SolarPanel class | Advaith |
| battery.h / .cpp | Battery class | Jailen |
| load_profile.h / .cpp | LoadProfile class | Advaith |
| energy_manager.h / .cpp | HourResult struct and EnergyManager class | Both |
| report.h / .cpp | Report class | Jailen |
| input_reader.h / .cpp | readHourlyCsv(), promptDouble() | Advaith |
| tasks.h / .cpp | SimulationState and one function per menu option (T1 to T9) | Owner of each task |
| tests.cpp | taskRunTests() (T10) and the class tests | Jailen |
| data/ | Sample sunlight and demand CSV files | Advaith |
| docs/ | Iteration report PDFs and the Excel project plan | Jailen |
| README.md | Build and run instructions | Jailen |

For Iteration 2 the code is kept in one file, solar_sim.cpp, with a comment banner marking where each planned file begins. This let us get the menu compiling quickly while we were still deciding on the class interfaces. At the start of Iteration 3 we will move each section into the files above, without changing any function names, and update the build command in the README.

## 3.5 Class Diagram

SimulationState holds one of each object and is passed to every task function. EnergyManager reads the panel and the load, updates the battery, and stores one HourResult per hour. Report only reads those results. A filled diamond means "holds," a dashed line means "uses."




# 4. Implementation Progress

We built the menu first, then wrote a function for every task, even if it only prints a message, so the program compiled from the start. Every class is declared with its data and methods, but most of the methods are still stubs. Each of us is now replacing only our own stubs.

Where each started task stands:

- T1 (Advaith): reads the hour,value rows from a CSV file and stores them. So far it only checks that the file opens. Checks for missing, negative, or out-of-order values still need to be added.

- T3 (Advaith): asks for the panel size and efficiency and stores them. energyAt() is written, but the hourly output table and the daily total are not.
- T4 (Jailen): asks for the battery capacity, minimum charge, and rate limit and stores them. charge() and discharge() are still stubs, and Jailen is still working out how to apply the rate limit and the minimum charge in the same function.
- All other options print "not implemented yet."

Build and run:

```
g++ -std=c++17 -Wall -Wextra -pedantic solar_sim.cpp -o solar_sim
./solar_sim
```

Sample run showing a started task and a stub:

```
Choose an option: 4
Enter battery capacity in kWh (1 to 100): 13.5
Enter minimum charge in % (0 to 50): 10
Enter max charge/discharge rate in kW (0.5 to 50): 5
Battery: 13.5 kWh capacity, 1.35 kWh minimum, 5 kW max rate

Choose an option: 5
T5 Run 24-hour simulation: not implemented yet.
```

Repository layout: the source code is in the root of the repository, sample inputs are in data/, and docs/ holds this report as a PDF and the Excel project plan. We are trying to commit small changes with short messages that name the task, for example "T4: add setLimits" or "T1: read values from CSV," instead of one large commit at the end.

# 5. What We Are Still Figuring Out

Since this is our first group project, there are several parts of the process we have not settled yet:

- **Git workflow:** we set up branches, but we are still learning how to review and merge each other's changes and how to avoid merge conflicts. Keeping all the code in one file makes conflicts more likely, which is one reason we plan to split it.
- **Task size:** we do not know yet how long each task will take, so the dates in the timeline are estimates. We will adjust them once we see how long the first classes take.
- **Testing:** each class needs at least three tests, but we have not decided whether tests should run from the menu, from a separate test program, or both.
- **Shared code:** we have not decided how the two of us will write EnergyManager together, for example whether we work on it side by side or split its functions.
- **Working as two people:** since Teammate 3 left, we are still checking whether the work is split fairly, and we will move tasks if one of us falls behind.

We plan to talk through each of these at our weekly meetings and write down what we decide in the meeting notes.

# 6. Revised Timeline

The phases match Iteration 1. Phase 2 now includes the system design and the menu, and LoadProfile and the Report stubs moved into Phase 2 because there are only two of us. Since we are new to this, we expect these dates to move, and the buffer week in late November gives us room for that.

| Phase | Dates | Work | Status |
|---|---|---|---|
| Phase 1: Planning | Through Oct 13 | Project plan, repository, class names | Done |
| Phase 2: Design and individual classes | Oct 14 to Nov 6 | System design, menu with stubs, SolarPanel, Battery, LoadProfile, input reader, first tests, sample files | In progress |
| Phase 3: Putting it together | Nov 7 to Nov 20 | EnergyManager routing, Report class, energy balance and battery limit tests, first full 24-hour run | Not started |
| Buffer and rework | Nov 21 to Nov 29 | Bug fixes and catch-up, Thanksgiving break (Nov 26) | Not started |
| Phase 4: Finishing | Nov 30 to Dec 8 | 7-day run, battery size comparison, invalid input tests, final report, presentation on Dec 8 at 10:18 A.M. | Not started |

# 7. Contribution Table

| Team member | Completed this iteration | Responsibilities next iteration |
|---|---|---|
| Advaith Nair | Built the menu and switch statement<br>Wrote the stub for every task<br>Started the CSV reader (T1) and the SolarPanel class (T3)<br>Made the first sample input files<br>Drafted the class and file structure design<br>Kept GitHub Issues up to date | Finish the CSV reader and its input checks (T1, T2)<br>Finish the SolarPanel and LoadProfile classes (T3)<br>Create the 7-day sample file<br>Start EnergyManager with Jailen (T5, T6, T9)<br>Keep GitHub Issues up to date<br>Review Jailen's code before merging |

| | | |
|---|---|---|
| Jailen Patel | Served as Project Leader for the first time<br>Started the Battery class (T4)<br>Wrote the Report class interface (T7, T8)<br>Made the first version of the task table and Excel plan<br>Drew the class diagram and put together this report<br>Kept the meeting notes | Finish charge() and discharge() (T4)<br>Decide on a testing approach and write the first tests (T10)<br>Start the Report class (T7, T8)<br>Start EnergyManager with Advaith (T5, T6, T9)<br>Update the README and meeting notes<br>Review Advaith's code before merging |

# 8. Risks and Next Steps

- Most of the code is still stubs, so Phase 2 has a lot left. We will finish T1 to T4 before starting anything shared.
- With two people, a busy week for one of us slows the whole project. The buffer week in late November is still in place, and the Project Leader will move tasks early if someone falls behind.
- Because we are still learning Git, we may lose time to merge problems. We will commit small changes and pull the latest code before starting work each day.
- Splitting the single file into headers could cause build errors. We will do it in one commit at the start of Iteration 3 and make sure the program still builds before any other work.